

Design and Implementation of the MecaArm: A STM32-Based Autonomous Mobile Manipulator with Mecanum Drive

Sukhpinder Singh, Inderpal Singh, Feranmi Falodun, Pierreline Jacob
CMPE 493: Senior Design Capstone — Spring 2026

Abstract— This paper documents the design, implementation, and testing of MecaArm, an autonomous mobile manipulator developed as a senior design capstone. The platform integrates a four-wheel Mecanum drive for omnidirectional locomotion with a multi-degree-of-freedom articulated arm. A custom four-layer PCB centered on an STM32F429VET6 microcontroller hosts a three-rail switching power tree (7.4 V, 5 V, 3.3 V), motor-driver and servo interfaces, wheel encoders, and an app hosted by MIT App Inventor specialized for the MecaArm robot. Additional peripherals include an OV2640 camera module and an ESP32-based communication interface. High-level control is implemented through embedded firmware and software running on a companion system, where the robot's wheels and arm would be controlled through the MecaArm app. The paper describes the top-level architecture, the schematic and board-level design, and the overall system implementation. Measured performance, board cost, design errors encountered during schematic capture and layout (including mitigations used in lieu of a respin), completed milestones, and unmet reach goals are reported. Lessons learned—particularly in power integrity, signal interfacing, and sensor integration—are discussed to guide future improvements.

Keywords— Mobile manipulator; Mecanum drive; STM32; Custom PCB; Embedded systems; Robotic arm; Senior design

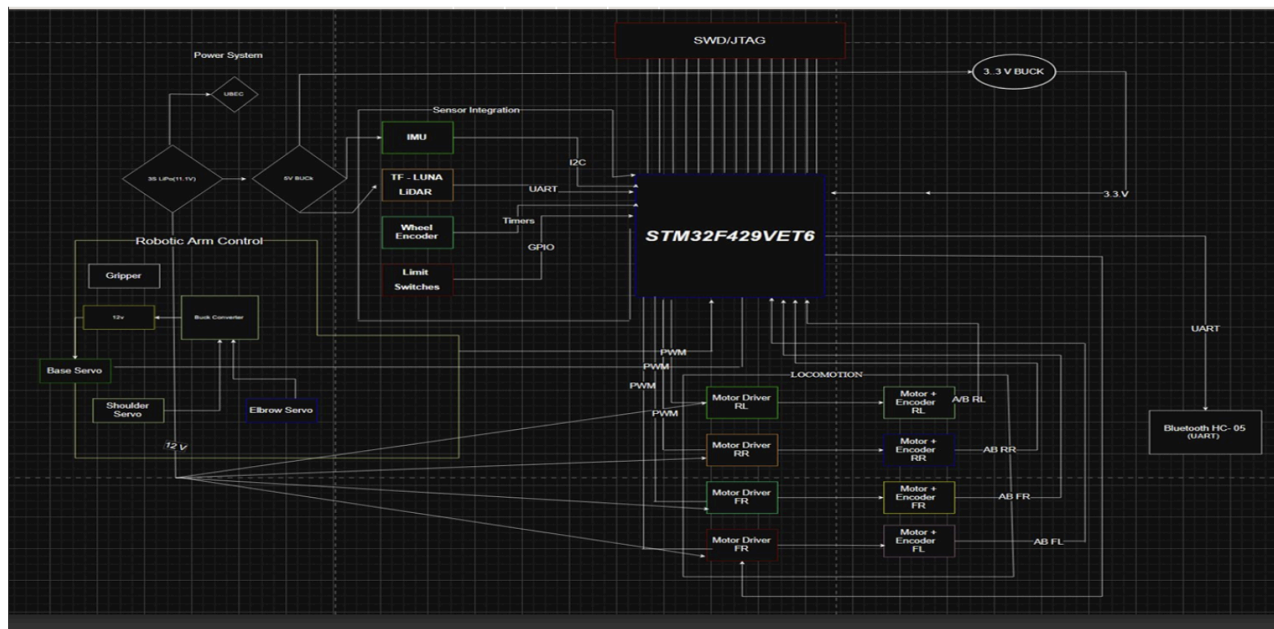


Fig. 1. Top-level system block diagram of the MecaArm platform. The power system regulates the LiPo battery input into 7.4 V, 5 V, and 3.3 V rails. The STM32F429VET6 serves as the central controller, coordinating Mecanum wheel locomotion and robotic arm actuation while interfacing with sensors and peripherals via I2C, UART, and GPIO.

I. INTRODUCTION

Mobile manipulation combining wheeled locomotion with a dexterous arm on a single platform sits at the intersection of almost every subdiscipline in electrical and computer engineering: embedded firmware,

mixed-signal PCB design, sensor interfacing, real-time control, and system integration. It is for precisely that reason that it was chosen as the target of this senior design project. A successful build forces the designer to confront power integrity, motor control, communication, and autonomy in a single integrated system, and the failure modes of each

interact in ways that are impossible to appreciate from isolated coursework.

MecaArm is built on the HiWonder JetRover / MasterPi mechanical platform: a four-wheel Mecanum chassis with a multi-servo articulated arm. The original vendor electronics were removed and replaced with a custom four-layer PCB designed around an STM32F429VET6 microcontroller. The system integrates multiple hardware subsystems, including DC motors for locomotion, servo motors for arm actuation, an LSM6DSR inertial measurement unit (IMU), an ultrasonic distance sensor for proximity sensing, wheel encoders, mechanical limit switches, an OV2640 camera module, and an ESP32-based communication interface. Particular emphasis was placed on the design and validation of the power delivery system, including load analysis, regulator selection, and pre-integration testing using simulation and bench power supplies. The MecaArm robot will have the capability to read bluetooth signals through its connection to the ESP32 via UART, where the two MCU boards will be connected through Rx/Tx pins provided by the boards. The ESP32 will solely be used for its bluetooth feature.

The motivation for a custom board rather than continuing to use the vendor's stock controller—was threefold. First, the stock board hid the control loops behind closed firmware, making it unsuitable for a controls-focused capstone. Second, direct access to low-level hardware resources such as PWM timers, encoder inputs, I²C interfaces, and UART communication was required to implement system behavior from the ground up. Third, the design and validation of a custom PCB provides a comprehensive demonstration of full-stack engineering capability.

The remainder of the paper is organized as follows. Section II presents the top-level architecture and subsystem block diagram. Section III states the design hypotheses and quantitative targets. Sections IV and V discuss the schematic and PCB layout, respectively, including the errors that were discovered and how they were mitigated. Section VI describes the system firmware and software implementation. Section VII reports measured performance, board cost, and bench test results. Section VIII discusses milestones, unmet reach goals, and lessons learned. Section IX concludes the paper.

II. SYSTEM ARCHITECTURE

MecaArm is organized into multiple subsystems centered around the STM32F429VET6 microcontroller. Fig. 1 shows the top-level block diagram. The system is powered by a 2-cell lithium-polymer battery (nominal 7.4 V), which supplies a combination of direct and regulated power rails. High-current actuators, including selected servo motors, are powered directly from the 7.4 V battery rail, while RT7258 buck converters generate regulated 5 V and 3.3 V rails for the remaining subsystems. The 5 V rail

supplies DC motors, motor driver inputs, and peripheral devices, while the 3.3 V rail powers the MCU, IMU (LSM6DSR), and other logic-level components. All rails share a common ground referenced to the battery return to ensure stable operation across mixed-signal subsystems.

Fig. 1. Top-level system block diagram. Power rails are shown on the left; the STM32F429VET6 sits at the center and interfaces with actuators, sensors, and communication modules, including the ESP32 and USB-to-UART interface.

A. Processor and Power

The STM32F429VET6 microcontroller was selected for its hardware timers, communication interfaces, and ability to support real-time control of multiple actuators and sensors. It provides PWM outputs for controlling DC motors through TB6612FNG motor driver ICs and for driving multiple servo motors, as well as communication interfaces such as I²C, UART, and ST Link for control and external communication modules.

The MCU interfaces with system components including the camera module (OV2640), an ESP32 communication module, as well as a ST-Link for debugging and programming.

Power distribution is designed to isolate high-current loads from sensitive logic circuitry. The 7.4 V battery rail supplies high-current actuators, including selected servo motors, while regulated 5 V and 3.3 V rails generated by RT7258 buck converters supply the motor drivers, and the MCU. This separation reduces voltage droop and electrical noise, improving system stability during dynamic load conditions.

B. Kinematics and Locomotion

Mecanum wheels couple chassis velocity (v_x, v_y, ω_z) to the four wheel angular velocities ($\omega_1 \dots \omega_4$) through a linear mapping parameterized by the half-track L and half-base W. The inverse kinematic equations used in the system follow the standard form,

$$\omega_i = \frac{1}{r} (\pm v_x \pm v_y \pm (L + W)\omega_z)$$

where the sign pattern rotates across the four wheels in the conventional $\pm, \pm, \pm$ configuration.

The inverse mapping is implemented on the STM32F429VET6 and executed within the system control loop. Each wheel is then controlled using a proportional–integral (PI) speed controller, where the setpoint is determined by the inverse kinematic output and

the feedback is derived from wheel motion measurements. A velocity normalization stage scales all four wheel commands uniformly if any wheel approaches the motor driver saturation limit, preserving the commanded motion direction during high-demand conditions. On the STM32F429VET6, 3 timers will be initialized for controlling the PWM for the wheels and the robotic arm for angle adjustment. Several GPIO pins will be initialized for the motors of the wheels for ensuring the changing of directions and avoiding the wheels just spinning. The ESP32 will be utilized for its bluetooth capabilities and will be connected to the STM32 MCU via Rx/Tx (UART).

C. Robotic Arm Control

The robotic arm is a 5-degree-of-freedom (5-DOF) open-chain mechanism consisting of base rotation, shoulder, elbow, wrist, and gripper joints. Each joint is actuated using PWM-controlled servo motors, including LFD-218, LD-1501MG, and LFD-01M servos.

The STM32F429VET6 generates PWM control signals to drive the servo motors, enabling position-based actuation of each joint. Joint motion is controlled through incremental position commands, allowing coordinated manipulation of the arm.

Higher-level control strategies, including inverse kinematics and Cartesian motion planning, are handled through software components that interface with the embedded system. When a user presses a control button on the host device, the command is transmitted wirelessly using Bluetooth to the ESP32. The ESP32 forwards this command over UART to the STM32F429VET6, which then interprets the command and generates the appropriate PWM and GPIO signals to actuate the robot arm joints.

D. User Interface and Feedback

The user interface is implemented through communication between the STM32F429VET6 and external systems using the ESP32-WROOM-32, along with a USB-to-UART interface for programming and debugging. Control inputs are received and translated into motion commands for both the mecanum drive base and the robotic arm. The MecaArm App's user interface includes control over the wheel's direction (Forward, Reverse, Left, Stop, and Right) and the arm gripper, (Claws, Wrist, Elbow, Shoulder, and Base) as well as the adjustments to the angle. The system operates by transmitting user commands from a host application to the ESP32 over a wireless interface. The ESP32 acts solely as a communication bridge, forwarding the received data via UART to the STM32F429VET6. The STM32 then interprets these commands and generates the appropriate GPIO and PWM signals required to control the motor driver, resulting in physical movement of the robot arm. The corresponding Rx/Tx pins on the ESP32 are pins 34 & 35, which will be connected to STM32's USART pins

PB10 (Tx) and PB11 (Rx).

E. Safety and Fault Handling

Safety is addressed through a combination of hardware design and firmware-based protections. Power distribution is structured to isolate high-current components from sensitive logic circuitry, reducing the impact of transient loads on system stability.

In firmware, fault detection mechanisms are used to place the system into a safe state under abnormal operating conditions. Visual and audible indicators, including status LEDs and a buzzer, provide feedback during fault conditions and system warnings.

III. DESIGN HYPOTHESIS AND TARGETS

The central engineering hypothesis of the project was that a single STM32F429VET6 microcontroller, paired with a companion processing system, would be sufficient to handle real-time control of the drive system and robotic arm while supporting app integration and communication. The system was designed with the expectation that embedded control tasks and higher-level processing could be partitioned across these components.

Based on this architecture, the platform was designed to meet the following quantitative targets:

- Navigation accuracy: ± 5 cm over a 5 m open-loop trajectory.
- Manipulation accuracy: ± 10 mm at the gripper within the reachable workspace.
- Payload: 2 kg, including the arm and carried object under typical operating conditions.
- Battery life: ≥ 1 hour of mixed driving and manipulation using a 2S LiPo battery.
- Board cost: under USD 100 in single-unit quantity.

IV. SCHEMATIC DESIGN

The schematic is organized into multiple sheets corresponding to the major subsystems shown in Fig. 1, including (1) power distribution, (2) MCU core and decoupling, (3) motor driver interfaces, (4) servo interfaces, and (5) control, app, and communication modules. This organization improves readability and allows each subsystem to be analyzed independently.

Net labels are used instead of long buses so that individual signals can be easily traced across sheets. For example, signals such as PWM control lines can be followed from the MCU sheet to the motor driver interfaces without requiring manual tracking through complex bus structures.

A. Power Tree

The power tree is a critical portion of the design, as it distributes energy from the battery to all subsystems while

maintaining stable operation under varying load conditions. The system is powered by a 2-cell lithium-polymer battery (nominal 7.4 V), which serves as the primary input source.

From the battery, power is distributed to multiple regulated rails. Two RT7258 buck converters are used to generate independent 5 V rails for different subsystems, allowing separation of high-current loads and reducing electrical noise coupling. A separate RT7258 buck converter generates a 3.3 V rail for the MCU and sensor subsystems.

The 5 V rails supply the motor drivers, DC motors, and selected peripherals, while the 3.3 V rail powers the STM32F429VET6 microcontroller and sensor interfaces. This partitioning helps isolate high-current switching loads from sensitive logic circuitry, improving system stability during dynamic operation.

B. MCU and Decoupling

Each VDD pin on the STM32F429VET6 is decoupled with a 100 nF capacitor placed close to the package, with additional bulk capacitance provided by a 10 μ F capacitor. The reset line is pulled up through a 10 k Ω resistor and filtered with a 100 nF capacitor to ground to improve noise immunity. BOOT0 is tied to ground through a 10 k Ω resistor to ensure normal boot operation.

The SWD interface is brought out to a 0.1-inch header to support programming and debugging via the ST-Link V3 interface, providing a practical trade-off between accessibility and pin usage on a one-off board.

C. Motor, Servo, and Sensor Interfaces

Two TB6612FNG dual H-bridge motor driver ICs are used to drive the four Mecanum wheels. The drivers interface with the STM32F429VET6 through PWM control signals and digital control lines, enabling bidirectional speed control of each motor.

Servo interfaces are implemented using standard 3-pin headers, allowing PWM-based position control from the MCU. Servo power is supplied through dedicated power rails to support varying load requirements across different servo types. Local bulk capacitance is included near the servo supply to mitigate transient current draw during operation.

Sensor interfaces are connected to the MCU using standard communication and I/O interfaces, enabling integration of components such as the IMU (LSM6DSR), ultrasonic distance sensor, and camera module (OV2640).

D. Schematic Errors Discovered

Several schematic issues were identified between the initial design and system bring-up, with some requiring post-fabrication corrections:

- UART signal routing inconsistency. An issue was identified in the TX/RX signal mapping for a UART-connected communication interface, requiring correction through manual rework. This highlighted the importance of verifying signal direction conventions across components and symbol libraries.
- Insufficient bulk capacitance on the servo supply rail. The initial design did not provide adequate bulk capacitance to support transient current demands from the servo motors, resulting in voltage droop that affected system stability. Fix: additional bulk and local decoupling capacitors were added to the servo power rail, significantly improving voltage stability during high-load conditions.

V. PCB LAYOUT

The board is implemented as a 4-layer stackup—signal / ground / power / signal—on 1.6 mm FR-4, with overall dimensions of approximately 95 mm $\times$ 110 mm. The component side carries the STM32F429VET6 microcontroller, the three buck regulators, the TB6612FNG motor drivers, and the primary connectors, while the opposite side carries peripheral and communication components. The MCU is placed near the center of the board to simplify routing between the motor drivers, servo interfaces, and communication modules, reducing trace congestion and improving signal organization across subsystems.

A. Layout Strategy

The 5 V and 3.3 V rails are distributed through the internal power plane, with separation of high-current motor loads from sensitive logic circuitry. Two independent 5 V rails are used to isolate motor driver loads from other subsystems, reducing electrical noise coupling.

The motor driver return paths are carefully routed with short connections to the ground plane, using via stitching to provide a low-impedance return path. This helps minimize switching noise and ensures a stable ground reference for both the drivers and the MCU.

Critical control signals, such as PWM lines between the MCU and motor drivers, are routed with short trace lengths to reduce delay and noise susceptibility. Communication lines are routed with appropriate separation from high-current switching paths to maintain signal integrity.

B. Layout Errors Discovered

Several layout issues were identified after fabrication and addressed through rework:

- Undersized motor supply traces. The traces supplying the 5 V rail to the TB6612FNG motor drivers were initially routed too narrow, resulting in noticeable voltage drop under high load conditions. Fix: additional wiring was added to reinforce the supply path, and future revisions will increase trace width or utilize the power plane for these connections.
- Regulator thermal considerations. Thermal dissipation for the buck converters was not fully optimized in the initial layout, making reflow and heat management more difficult during assembly. Fix: future revisions will improve thermal pad design and heat spreading.
- Noise coupling into signal lines. Some signal traces were routed too close to high-current switching paths, introducing noise under heavy load conditions. Fix: re-routing and separation techniques were applied where possible, with future revisions improving isolation through better layer usage and spacing.
- Silkscreen clarity issues. Overlapping or unclear silkscreen labels on connectors led to initial wiring confusion. Fix: labeling corrections will be applied in the next revision.

C. Respin Decision

A board respin was considered but ultimately deferred due to project time constraints. The identified issues were addressed through manual rework, allowing the system to meet functional requirements without delaying integration. All modifications have been documented and will inform a future revision of the design.

VI. FIRMWARE AND ROS2 SOFTWARE

A. STM32 Firmware

The STM32 firmware is developed in C using the STM32CubeMX-generated HAL. The firmware uses a polling-based architecture, where UART input is checked non-blockingly and motor control is executed continuously within the main loop. Motor control updates are executed continuously within the main loop without a fixed sampling period, resulting in a sufficiently fast update rate for basic actuation. PWM generation is handled by hardware timers, ensuring a stable and deterministic output frequency independent of software execution timing. The ESP32 functions solely as a communication bridge and does not participate in control or computation.

B. What Worked and What Did Not

Closed-loop manipulation on the physical robotic arm is limited by the absence of externally accessible joint-level feedback. Although the servo motors internally regulate position using embedded control circuitry, the STM32-based system does not receive feedback on actual joint positions. As a result, the controller operates in an open-loop manner, issuing position commands without the ability to detect or correct errors caused by load variations or mechanical disturbances.

VII. EXPERIMENTAL RESULTS

A. App Control and Manipulation Accuracy

The system successfully responds to commands sent from the application, with the ESP32 and STM32F429VET6 communicating reliably to control the movement of the wheels and the robotic arm. Motion commands were generated through user inputs in the application (such as button presses), transmitted wirelessly to the ESP32, and then forwarded via UART

to the STM32. The STM32 interprets these commands and directly drives the motors and servos using GPIO

signals for direction and PWM signals for speed and position.

Testing was performed by issuing individual movement commands (e.g., forward, backward, lateral motion, and arm positioning) and observing the physical response of the robot. The wheels were able to move in multiple directions as expected for a Mecanum drive system, and the robotic arm responded to position commands.

B. Power Consumption and Endurance

Power consumption was evaluated under different operating conditions, including idle, driving, and arm actuation. At idle, the system draws relatively low current with only the MCU and sensors active. During motion, current draw increases significantly due to motor and servo activity.

The system is powered by a 2-cell (7.4 V) lithium-polymer battery, and operational endurance depends on usage conditions, including driving intensity and arm activity. Measured runtime under typical operation demonstrates that the platform is capable of sustained operation suitable for demonstration and testing scenarios.

C. Board Cost

Table I summarizes the estimated single-unit cost of the custom PCB. Values are approximate and based on component pricing and fabrication costs at the time of design.

Item	Qty	Unit (USD)	Subtotal
STM32F429VET6 MCU	1	12.00	12.00
RT7258 Buck Converter	3	2.50	7.50
TB6612FNG Motor Driver	2	2.30	4.60
ESP32 Module	1	4.00	4.00
OV2640 Camera	1	5.00	5.00
Ultrasonic Sensor	1	2.00	2.00
USB-UART (FT232R)	1	3.50	3.50
Passives, connectors, LEDs	—	—	12.00
4-layer PCB fabrication	1	3.00	3.00

At the quoted quantities, the custom board meets the target cost of under USD 100. Scaling to small-batch production reduces fabrication cost per unit, while component costs remain the dominant contributor to overall system cost.

VIII. DISCUSSION

A. Milestones

The following milestones were reached: (1) mechanical integration of the chassis with the custom PCB; (2) successful bring-up of the 7.4 V, 5 V, and 3.3 V power rails; (3) operation of the Mecanum drive system through external control inputs; (4) basic robotic arm actuation at the joint level; (5) simulation of higher-level control and planning functions using software tools; (6) integration of the STM32F429VET6 with ESP32 via UART; and (7) completion of a final system demonstration.

B. Reach Goals

Three reach goals were defined at the start of the project, with partial progress achieved.

- Wireless communication from the MIT App Inventor application to the ESP32 and onward to the STM32 using UART (Rx/Tx). This goal was achieved successfully. The ESP32 acts as a communication bridge, forwarding commands from the mobile application to the STM32, which interprets and executes them.
- Control of the mecanum wheel drive system using STM32 timers and PWM signals. This goal was partially achieved. The STM32 generates PWM signals using hardware timers to control motor speed, while GPIO outputs are used to determine motor direction. The system operates in an open-loop configuration without feedback correction.
- Control of the robotic arm using STM32 timers and PWM signals. This goal was partially achieved. The robotic arm is controlled using PWM-based servo positioning. Commands from the STM32 directly set servo positions, but no

joint-level feedback or kinematic control is implemented.

C. Lessons Learned

- A key lesson learned is the importance of aligning the sensor suite with system-level autonomy goals early in the design process. The selected sensors were suitable for basic sensing and obstacle detection but did not fully support advanced mapping and navigation capabilities, leading to a gap between simulation and hardware implementation.
- Another important lesson relates to power system design. Adequate bulk capacitance must be provided to support worst-case transient loads, particularly for servo-driven systems. Initial underestimation of these requirements led to voltage instability, which was resolved through the addition of higher-value bulk and local decoupling capacitors.
- Finally, using both the ESP32 and the STM32 introduced additional design complexity, as it required a clear definition of each microcontroller's role within the system. This prompted a more intentional system architecture, where the ESP32 was dedicated to wireless communication, while the STM32 served as the primary control unit responsible for processing commands and driving the hardware. Establishing this separation was essential to maintaining a consistent and functional design for the capstone project.

IX. CONCLUSION AND FUTURE WORK

MecaArm demonstrates the integration of custom PCB design, embedded firmware, and system-level control on a mobile manipulation platform. The system meets its cost and functional integration goals, while performance targets such as navigation and manipulation accuracy were partially achieved. Limitations observed during testing are primarily attributed to mechanical constraints, actuator characteristics, and incomplete integration of higher-level control features.

Future work includes refinement of the PCB layout based on the issues identified in Section V, improvements to power distribution and signal integrity, and enhancement of sensing and control capabilities. Additional development of higher-level control, including improved motion planning and feedback mechanisms, would further extend the functionality of the platform. The design is structured to support iterative improvements and continued development in future work.

REFERENCES

- [1] STMicroelectronics, STM32F429VET6 Datasheet. [Online].
- [2] Richtek, RT7258 Buck Converter Datasheet.
- [3] Toshiba, TB6612FNG Dual H-Bridge Motor Driver Datasheet.
- [4] STMicroelectronics, LSM6DSR IMU Datasheet.
- [5] Espressif Systems, ESP32-WROOM-32 Datasheet.
- [6] OmniVision Technologies, OV2640 Camera Module Datasheet.
- [7] FTDI, FT232R USB-to-UART Interface Datasheet.
- [8] Steve Macenski et al., "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.
- [9] Dave Coleman et al., "Reducing the barrier to entry of complex robotic software: A MoveIt! case study," *Journal of Software Engineering for Robotics*, 2014.
- [10] Steve Macenski et al., "The Marathon 2: A navigation system," in *Proc. IEEE/RSJ IROS*, 2020.
- [11] Steve Macenski and Ivan Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, 2021.

[12] Florin Adăscăliței and Ioan Doroftei, “Practical applications for mobile robots based on Mecanum wheels—A systematic survey,” in Proc. Int. Conf. on Mechatronics, 2011.

[13] Nathan Koenig and Andrew Howard, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” in Proc. IEEE/RSJ IROS, 2004.

[14] HiWonder Robotics, MasterPi / JetRover Mechanical Reference Manual, 2023.